

AI335L Deep Learning Lab

Experiment No. 08

Deadline: Apr 27, 2026 11:00 AM (Before Class)

Lab Task

Demonstrate your ability to create a comprehensive SRS Documentation for your Semester Project while following the below given instructions

Project Roadmap & Documentation

1. Welcome & Context

Over the next Six weeks, you will design, build, evaluate, and deploy a deep learning system that demonstrates mastery of every major topic covered in this course. **A project of this scale lives or dies on the quality of its planning.** This first week is dedicated entirely to that, to producing a well-researched roadmap that becomes the blueprint for everything you do afterwards.

When this document is finished, you should be able to answer, without hesitation: *What am I building? Why does it matter? Which architectures will I use, and where? What data do I need? How will I know if I succeeded?* If any of those answers are still vague, your roadmap is not yet done.

A note before we begin: this is not a “filler” assignment. Every section below exists because students who skipped it in past semesters lost time later, usually in week 4 or 5, when it is far too late to recover.

2. Learning Objectives

By the end of Phase 1, you will be able to:

- Frame a real-world problem as a well-defined deep learning task
- Conduct a focused literature review and position your work against prior art
- Plan a multi-architecture DL system end-to-end
- Justify architectural choices with technical reasoning, not vibes
- Communicate a technical plan clearly in writing

3. Project Requirements (What Your Final System Must Cover)

Your final project (delivered in Phase 6) must integrate the following deep learning topics. Your Phase 1 roadmap must explain *where* and *how* each will appear in your system. You do not need every topic as the core model, but each must be integrated meaningfully, not name-dropped.

3.1. Architectures (use at least three of these in a substantive way)

- Feedforward Neural Networks (MLP/ANN) — at minimum as a baseline
- Convolutional Neural Networks (CNNs)
- Recurrent architectures (RNN, LSTM, or GRU)
- Transformer or attention-based models
- A generative model (Autoencoder, Variational Autoencoder, or GAN)

3.2. Required techniques (all of these)

- Transfer learning with at least one pretrained model
- Regularization (combination of Dropout, BatchNorm, L1/L2, Early Stopping)
- Comparison of at least two optimizers (e.g., SGD vs. Adam vs. RMSProp)
- Thoughtful data preprocessing and augmentation
- A defined hyperparameter tuning strategy (Grid Search, Random Search, Bayesian, or Optuna)
- A model deployment plan (Streamlit, Gradio, Flask, FastAPI, or a small web app)
- A reproducibility plan (random seeds, version pinning, environment file)

3.3. Project type (pick one direction)

You may choose any application domain (computer vision, NLP, audio, time series, multimodal, healthcare, finance, agriculture, climate, accessibility, etc.) The only rule is that the problem must genuinely benefit from deep learning (no projects where a logistic regression would clearly do the job equally well).

4. Required Sections of the Roadmap Document

The following structure is mandatory. Use these section names exactly so I can grade consistently across teams.

Section 1: Cover Page

Project title, team members (with student Reg. IDs and names), course code, instructor name, semester, and submission date. Add one short tagline (one sentence) describing the project.

Section 2: Executive Summary / Abstract (≈ 250 words)

A standalone, self-contained summary covering: the problem, why it matters, your proposed approach, the deep learning components involved, the data you will use, and the expected outcome. Write this last, **after the rest of the document is done**.

Section 3: Problem Statement & Motivation (1.5–2 pages)

Clearly articulate the problem you are solving. Explain who is affected, what currently exists, and where existing solutions fall short. Cite real numbers where possible (market size, prevalence, accuracy gaps in current tools, etc.). Finish with a sharp one-paragraph statement of the gap your project addresses.

Section 4: Literature Review (3–4 pages, minimum 6 references)

Survey what others have done. Include at least:

- Two foundational papers for the core architecture you plan to use
- Two recent papers (within the last 3 years) on the same problem or domain
- Two additional sources of your choice (papers, technical blogs from reputable labs, official documentation)

For each work, **summarise**:

- the problem they solved,
- the approach they took,
- the dataset they used,
- their reported results, and
- the limitation that leaves room for your project.

End the section with a comparison table.

Section 5: Project Objectives & Scope

Use the **SMART framework**, every objective must be **S**pecific, **M**easurable, **A**chievable, **R**elevant, and **T**ime-bound. Aim for 3–5 primary objectives. Then explicitly state what is *out of scope*. Defining scope clearly is half the battle; vague scope is the most common reason projects fail in week 5.

Section 6: Dataset Description

Cover all of the following:

- Data source and licensing (must be legally usable)
- Size, format, and modality
- Class distribution (include a chart if classification)
- Quality issues you anticipate (imbalance, noise, missing values, label noise)
- Preprocessing pipeline you intend to apply
- Augmentation strategy
- Train / validation / test split strategy and rationale
- Backup dataset in case the primary one fails (this is mandatory, projects have died because a dataset disappeared)

Section 7: Proposed Methodology

The technical heart of the document. Include:

- A high-level pipeline diagram (**data** → **preprocessing** → **model** → **output** → **deployment**)
- For each architecture you plan to use: which one, why it fits the problem, and what role it plays in the system
- Justification for transfer learning choices, which pretrained model, why, what you will freeze, what you will fine-tune
- Loss function selection and reasoning
- Optimizer comparison plan
- Regularization plan
- Hyperparameter search strategy

A diagram is required, not optional. Hand-drawn and photographed is fine if it's clear.

Section 8: System Architecture

A second diagram showing the deployed system: how a user input flows through the application, the model, and back to a UI. Identify which component handles which responsibility (frontend, API, model server, storage).

Section 9: Technology Stack

A table listing every framework, library, and tool you will use, with the version you intend to pin and a one-line justification. Include hardware (Colab, Kaggle, local GPU, university lab) and where checkpoints will be stored.

Section 10: Six-Week Timeline

A week-by-week breakdown with deliverables per week, owner per task, and dependencies between tasks. A Gantt chart is strongly preferred. Build in at least one buffer day per week, projects always run over.

A reasonable shape, which you may adapt:

- **Week 1:** Roadmap (this assignment)
- **Week 2:** Data acquisition, EDA, preprocessing pipeline, baseline MLP
- **Week 3:** Core architecture (CNN/RNN/Transformer) implementation and training
- **Week 4:** Generative component, transfer learning, hyperparameter tuning
- **Week 5:** Evaluation, ablation studies, error analysis
- **Week 6:** Deployment, final report, presentation

Section 11: Evaluation Metrics

State which metrics you will use and *why each one is appropriate for your problem*. Going beyond accuracy is expected, for imbalanced classification, accuracy alone is misleading; for generation tasks you'll need

FID/BLEU/perplexity as appropriate. Include at least one qualitative evaluation method (sample outputs, confusion matrix analysis, attention visualization, etc.).

Section 12: Expected Outcomes & Deliverables

What does “done” look like? List every artifact: trained model weights, source code repository, deployed demo URL, final report, presentation slides, demo video.

Section 13: Risk Analysis & Mitigation

A table with at least 6 risks. For each: probability (Low/Med/High), impact (Low/Med/High), and a concrete mitigation plan. Common risks worth thinking through: insufficient compute, dataset issues, model not converging, team member illness, scope creep, integration failures during deployment.

Section 14: Team Roles & Responsibilities

Who owns what. Every major area should have a primary owner *and* a secondary reviewer. “We’ll all do everything together” is not a plan, it is how things fall through cracks.

Section 15: References

IEEE or APA format, consistent throughout. Minimum six references; more is welcome.

Section 16: Appendices (optional but encouraged)

Initial dataset samples, preliminary code snippets, environment YAML, anything else useful.

5. Submission Guidelines

- Format: PDF, single file, named Phase1_<TeamName>_Roadmap.pdf
- Include a public GitHub repository link on the cover page (the repo can be mostly empty at this stage, just a README, .gitignore, and folder structure)
- Submit on the LMS by the deadline announced in class
- Late penalty: 10% per day,

6. Evaluation Rubric

Criterion	Weight
Problem framing & motivation	10%
Literature review depth and quality	15%
Methodology and architecture justification	25%

Criterion	Weight
Dataset plan (including backup and ethics)	10%
Timeline realism and risk analysis	15%
Coverage of required DL topics	15%
Document quality (writing, diagrams, references)	10%

A roadmap that has all 16 sections but is shallow in Sections 4 and 7 will score lower than a tighter one with strong technical reasoning. **Depth beats length.**

9. What Comes Next

Phase 2 begins immediately after submission. You will start building data pipelines and your baseline MLP. The clearer your roadmap, the smoother that week will be.

Good luck. Build something you'll be proud to show.